

ProjectV — Сборник контрольных точек

КТ-2.1, КТ-2.2, КТ-3.1, КТ-3.2

2026-06-13

Содержание

1. Общая схема архитектуры (System Context / Container Diagram)	3
2. Стек технологий с обоснованием	5
3. Структуры данных	5
4. AppState — корневой контейнер	6
5. Описание ключевых программных модулей	7
6. Описание ключевых алгоритмов	8
7. План и виды тестирования	9
8. Сводный дашборд	9
9. Детальные тест-кейсы	9
9.1 ТС-01. Загрузка валидного glTF-фикстуры (Box) [Позитивный]	9
9.2 ТС-02. Одиночная ячейка жидкости падает ровно на 1 клетку за тик [Позитивный]	10
9.3 ТС-03. Сохранение и загрузка snapshot восстанавливают мир побайтово [Позитивный]	10
9.4 ТС-04. Изменение воксела на границе 8 чанков помечает все 8 [Позитивный]	10
9.5 ТС-05. ААВВ внутри фрустума помечается как видимый [Позитивный]	11
9.6 ТС-06. BMP-screenshot содержит корректный заголовок и пиксели [Позитивный]	11
9.7 ТС-07. Загрузка несуществующего glTF-файла возвращает ошибку, без segfault [Негативный]	11
9.8 ТС-08. Манифест с невалидным transform-суффиксом отбрасывается [Негативный]	12
9.9 ТС-09. Жидкость не разрушает платформу под собой [Негативный]	12
9.10 ТС-10. Wake пустого ассета возвращает ошибку и пустой результат [Негативный]	12
9.11 ТС-11. ААВВ за камерой (positive Z) отсекается фрустумом [Негативный]	13
9.12 ТС-12. Невалидные параметры CSM-сплитов клампятся в безопасный диапазон [Негативный]	13
10. Обзор остальных модулей	13
11. Реестр дефектов	14
12. ЧАСТЬ 1: Руководство администратора	16
12.1 Системные требования	16
12.2 Инструкция по развёртыванию (Linux)	16
12.3 Инструкция по развёртыванию (Windows)	16
12.4 Переменные окружения	17
12.5 Пресеты сцен	17

13. ЧАСТЬ 2: Руководство пользователя	24
13.1 Обзор интерфейса	24
13.2 Управление	24
13.3 Типовые сценарии (How-to)	25
13.3.1 Как запустить сцену VoxelLab?	25
13.3.2 Как разместить воксель?	25
13.3.3 Как удалить воксель?	25
13.3.4 Как перезагрузить шейдеры без перезапуска?	25
13.3.5 Как переключить пресет сцены?	25
13.3.6 Как снять capture для профилирования?	25
13.3.7 Как включить Vulkan validation layers?	25
13.4 FAQ	25
14. Статус готовности и сравнение с ТЗ	27
15. Архитектурные и инженерные вызовы (Post-mortem)	27
16. Организация рабочего процесса	28
17. Рефлексия и приобретённый опыт	28
18. План дальнейшего развития (Roadmap)	29

1. Общая схема архитектуры (System Context / Container Diagram)

ProjectV — десктопное приложение реального времени (нативный бинарник под Windows и Linux). Архитектура однопроцессная, не клиент-серверная: вся логика, рендер и физика выполняются в одном процессе на стороне пользователя.

Подсистемы первого уровня:

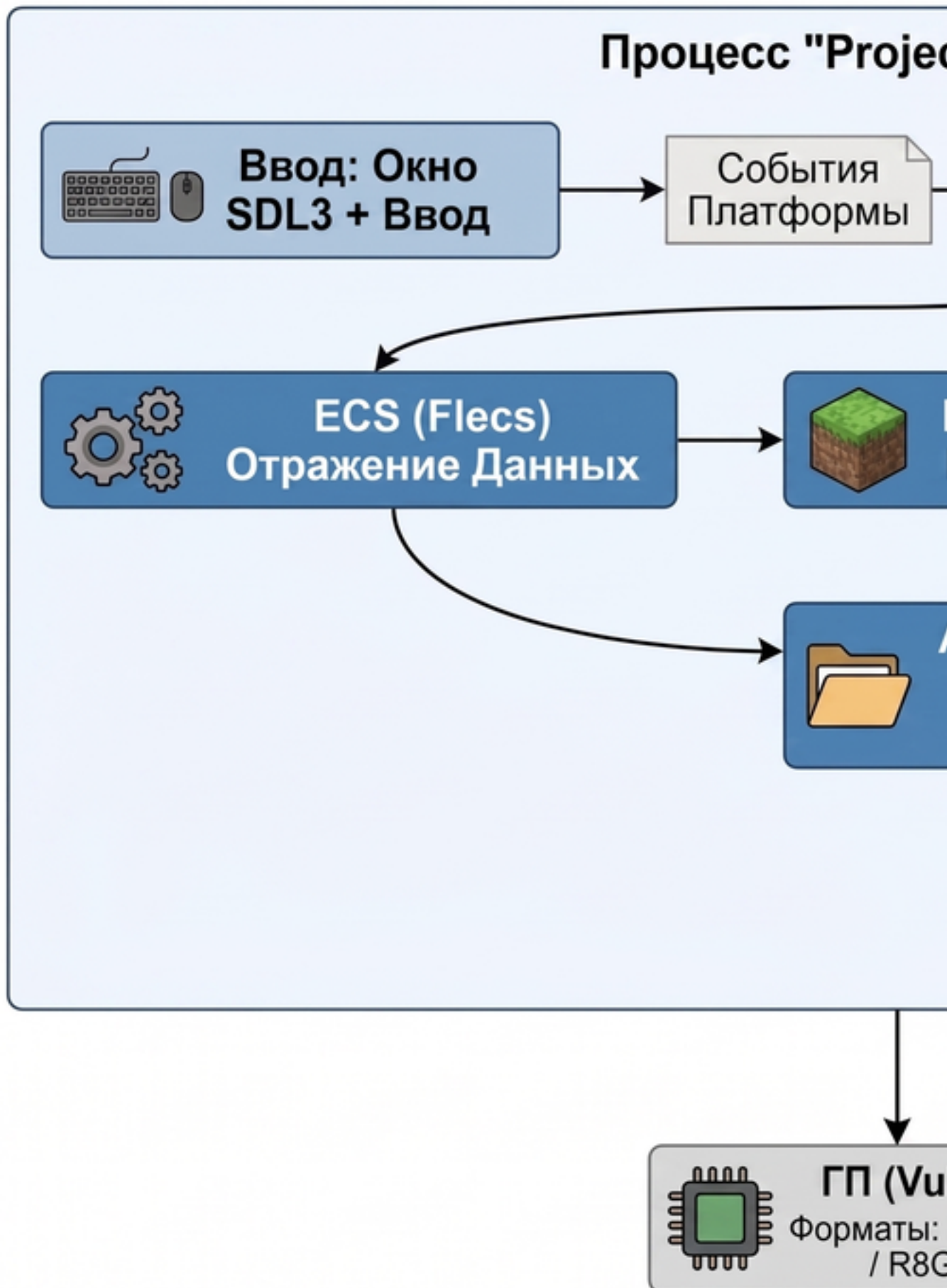
Подсистема	Зона ответственности	Зависимости
Платформа	Окно (SDL3), ввод, жизненный цикл	SDL3
Воксельный мир	Источник истины для всех воксельных данных	—
Физика	Walk-персонаж, столкновения со статической геометрией	Jolt
Аудио	Фоновая музыка, плейлист, горячая замена треков	miniaudio
ECS	Зеркало состояния мира для запросов и симуляций	flecs
Реестр ассетов	glTF / Draco / meshopt-ассеты, горячая перезагрузка	fastgltf, draco, meshopt
Рендерер	Мешинг, освещение, тени, TAA, ray march, debug-оверлей	Vulkan 1.4
GPU	Все вычислительные и графические ресурсы	—

Поток данных (упрощённо): SDL3 → события платформы → InputState → AppState → VoxelWorld (источник истины) → Физика (Jolt) + ECS (зеркало) + Реестр ассетов → Рендерер (подготовка на CPU) → Vulkan → GPU.

Графическая схема архитектуры (System Context / Container Diagram):

Архитектура "Project"

Процесс "Project"



2. Стек технологий с обоснованием

Языки и сборка:

Технология	Версия	Назначение
C++26	ISO/IEC 14882:2026	Основной язык реализации
C23	ISO/IEC 9899:2024	Подмодули с C-API (Jolt, draco, meshopt)
CMake	3.30+	Система сборки
Ninja	1.13+	Бэкенд сборки (быстрые инкрементальные сборки)
Clang 22	22.1.6	Компилятор (мультиплатформенный, C++26-фичи)

Платформа и графика:

Технология	Версия	Назначение
SDL3	3.4.10	Платформенный слой (окно, ввод)
Vulkan 1.4	1.4.350	Графический API
volk	pinned	Vulkan-загрузчик (runtime-загрузка)
VMA	pinned b3cbbb43	Менеджер памяти GPU (chronos-group)
GLSL → SPIR-V	4.5 / 1.4	Шейдеры (автоматическая компиляция через glslc)
RenderDoc	1.30+	GPU-отладчик

Сторонние библиотеки:

Технология	Версия	Назначение
flecs	2.2.0	ECS (header-only, MIT)
JoltPhysics	pinned	Физика (детерминированная)
Tracy	0.11+	Профилирование (CPU + GPU-зоны)
fmt	pinned	Типобезопасное форматирование
glm	pinned	Математика (header-only)
fastgltf	pinned	glTF-загрузчик (header-only, simdjson backend)
draco	pinned	Сжатие мешей (Google decoder-only)
meshoptimizer	pinned	Оптимизация мешей (vertex cache, перерисовка)
miniaudio	pinned	Аудио (header-only MIT, single-file)
nlohmann/json	3.11.3 (FetchContent)	JSON-парсинг (header-only MIT)
Dear ImGui	opt-in (OFF по умолчанию)	ImGui UI

Итого: 22 подмодуля + 1 FetchContent (nlohmann/json) + системные библиотеки (zlib, png, jpeg для текстур — opt-in).

3. Структуры данных

Структура	Поля (типы)
VoxelWorld	voxels: vector<uint8_t>, chunks: vector<...>
VoxelChunk	min/maxExclusive: Int3, rebuildQueued: bool, nonAirVoxelCount: Int
VoxelMaterial (enum)	Air=0, Glass=1, Fluid=2, FloorWhite=3, FloorGray=4 (uint8)
VoxelMaterialVisual (struct)	baseColor: vec4, surface: {A0, roughness, metallic, reflectance}
AppState (root)	см. §4
WorldState	voxelWorld: unique_ptr<VoxelWorld>, requestedScenePreset: VoxelScene
SceneResources	chunkDescriptors: vector<...>, sceneChunkVoxelData: ...
InteractionState	selection: RaycastHit, placementVoxel: Int3, hitNormal: vec3, hitDistance: float
PackedSceneChunkDescriptor	64 B std430 — см. детали ниже
PackedSceneVoxelFace	16 B — см. детали ниже
VoxelSceneLighting	9 × array<float, 4> — см. детали ниже

Детализация критичных структур:

- **PackedSceneChunkDescriptor** (64 B, std430, см. src/core/Types.hpp:110-118):
 - chunkOrigin: array<int32, 4> (offset 0) — начало чанка в мировых координатах;
 - chunkExtentAndNonAir: array<uint32, 4> (offset 16) — размеры + счётчик non-air;
 - voxelDataInfo: array<uint32, 4> (offset 32) — индексы в sceneChunkVoxelData;
 - drawRanges: array<uint32, 4> (offset 48) — диапазоны draw call'ов.
- **PackedSceneVoxelFace** (16 B, см. src/core/Types.hpp:96-108):
 - voxelCoord, axisMask, materialIndex, lightingData, packedExtents: uint32_t — координаты вокселя, маска осей, материал, освещение, packed extents.
- **VoxelSceneLighting** (9 × vec4, контракт с GLSL SceneLightingBuffer):
 - skyColorAndFogDensity, horizonColorAndFogStart, groundColorAndFogMax, sunColorAndIntensity, sunDirectionAndWrap, postProcess, sunShadowParams, sunContactShadowParams, ambientOcclusionParams.

Связи (FK):

- VoxelWorld → VoxelChunk (1:N) — каждый чанк принадлежит одному миру.
- VoxelChunk ↔ VoxelWorld (1:1) — границы (min/max) задают подмножество мировых координат.
- VoxelMaterialVisual → VoxelMaterial (N:1) — несколько визуалов могут ссылаться на один материал.
- WorldState → VoxelWorld (1:1) — unique_ptr.
- SceneResources → VoxelChunk (1:1) — chunkDescriptors[i] соответствует chunks[i].
- PackedSceneChunkDescriptor → VoxelChunk.min (1:1) — chunkOrigin задаёт начало чанка.
- PackedSceneVoxelFace → VoxelWorld.voxels (N:1) — грань ссылается на конкретный воксель.
- VoxelSceneLighting — общий контракт с шейдерами (GLSL SceneLightingBuffer).

4. AppState — корневой контейнер

AppState — единый корневой объект, через который проходят все данные между подсистемами.

Поле	Тип
platform	PlatformState
context	VulkanContextState
swapchain	SwapchainState
world	WorldState
render	RenderState
input	InputState
interaction	InteractionState
simulation	SimulationState
ecs	EcsStatePtr (unique_ptr<flecs::world>)
physics	PhysicsStatePtr (unique_ptr<PhysicsWorld>)
audio	AudioEnginePtr (unique_ptr<AudioEngine>)
benchmark	BenchmarkAutomationState
lookDevCapture	LookDevCaptureAutomationState
frame	FrameState

Краткое назначение полей:

- platform — окно SDL3 и обработчик изменения размера.
- context — VkInstance, Device, очередь, аллокатор.
- swapchain — VkSwapchainKHR, представления изображений, extent.

- world — requestedScenePreset, указатель на VoxelWorld.
- render — sceneResources, frameData, отладочные оверлеи.
- input — дельта мыши, снимок действий, replay.
- interaction — попадание raycast, размещаемый воксель, нормаль и дистанция попадания.
- simulation — аккумулятор, timeScale, frameStep.
- ecs — мир flecs (ECS-зеркало).
- physics — мир Jolt.
- audio — аудиодвижок с пользовательским удалителем.
- benchmark — frameCount, средние, минимум/максимум.
- lookDevCapture — прогрев, список видов для съёмки.
- frame — профиль CPU, тайминги рендера.

Связи между ключевыми сущностями:

Связь	Кард. FK	Назначение
VoxelWorld → VoxelChunk	1:N editVersion	версия мира (инкремент при правке)
VoxelChunk ↔ границы	1:1 min/max	границы внутри VoxelWorld
SceneResources → VoxelChunk	1:1 chunkDescriptors[i]	индекс чанка в chunks
PhysicsWorld ↔ VoxelWorld	1:1 (read-only)	физ-мир из solid-вокселей
Renderer → SceneResources	1:1 —	рендер читает стейтинг
CharacterVirtual (Jolt) → AppState	1:1 —	корневой контейнер

5. Описание ключевых программных модулей

ProjectV декомпозирован на 11 независимых модулей:

Модуль	Зона ответственности
Платформа	Окно (SDL3), обработка событий, InputState
Воксельный мир	Хранение, редактирование, грязные чанки, raycast
Физика	Walk-персонаж (CharacterVirtual), Jolt, синхронизация
Аудио	Плейлист, 5-секундное автообновление, горячая замена
ECS	Зеркало мира для запросов (MIT/RAII)
Реестр ассетов	Конвейер glTF / Draco / meshopt, горячая перезагрузка
Рендерер (подготовка на CPU)	Мешинг, освещение, данные кадра
Проекция теней	Тень по сцене, CSM, контактная тень
Ray March	Параллельный GPU-проход для объёмного рендера
ТАА	Temporal Anti-Aliasing, исправление гонки дескрипторов
Vulkan (низкий уровень)	Instance, Device, Swapchain, конвейеры, командные буферы

Ключевые файлы и типы по модулям (для быстрой навигации):

1. Платформа — src/platform/PlatformEvents.cpp, InputState.hpp
2. Воксельный мир — src/voxel/VoxelWorld.cpp, VoxelWorld.hpp
3. Физика — src/physics/PhysicsWorld.cpp, PhysicsWorld.hpp
4. Аудио — src/audio/AudioEngine.cpp, AudioEngine.hpp
5. ECS — src/ecs/EcsState.*
6. Реестр ассетов — src/asset/AssetManifest.cpp, AssetManifest.hpp
7. Рендерер (подготовка на CPU) — src/render/SceneResources.cpp, SceneResources.hpp
8. Проекция теней — src/render/ShadowProj.*
9. Ray March — src/render/RayMarchPass.*
10. ТАА — src/render/Taa.*
11. Vulkan (низкий уровень) — src/render/vulkan/*

6. Описание ключевых алгоритмов

1. **Игровой цикл** (AppMain). Физика с фиксированным шагом (60 Гц), рендер с переменным шагом, профилирование через Tracy.
2. **Greedy-мешинг** (voxel_mesh.comp). Объединение соседних solid-вокселей в четырёхугольные грани для снижения перерисовки и indirect-draw.
3. **Полномочия на walk** (Физика + Воксельный мир). CharacterVirtual синхронизирован с VoxelWorld.editVersion; только физика запрашивает столкновения, остальные подсистемы читают VoxelWorld read-only.
4. **Тень по сцене** (Проекция теней + voxel_mesh.comp). Каскадные карты теней (CSM) рассчитаны по sceneChunkVoxelData, а не по пирамиде видимости — оптимизация под плотные воксельные сцены.
5. **Клеточный автомат жидкости** (voxel_fluid.comp). Двусторонний обмен с VoxelWorld для материала Fluid=2: каждый шаг СА читает соседние ячейки, при необходимости обновляет значение.

7. План и виды тестирования

Проведено ручное функциональное тестирование runtime приложения и автоматизированное unit-тестирование всех программных модулей ProjectV. Тесты написаны на собственном лёгком фреймворке (tests/): функция `void TestXxx(TestContext &context)`, регистрируется через `RegisterTest(...)` (см. tests/CMakeLists.txt).

Виды тестирования:

- **Unit-тестирование** — ctest запускает 12 suites, ~140 test-функций (VoxelWorld 22, FluidCA 10, AssetLoader 10, FrustumCulling 5, BoxUvFixture 2, MeshBaker 4, DracoDecoder 3, Math 1, StringId 1, SunShadowCascadeSplits 1, ModuleSmoke 1, StdModuleProbe 1, плюс ProbeTest и CFrustumCullingTests).
- **Runtime smoke** — `tools/linux/Invoke-ProjectVRuntimeSmoke.sh` (Linux) / `tools/windows/Invoke-ProjectVRuntimeSmoke.ps1` (Windows) запускает runtime, генерирует 6 captures (FINAL, SHDW, CSM, CTSH, AOCC, LOCL) с sidcar-метаданными.
- **Профилирование** — Tracy CPU + GPU zones, sidcar-замеры (frame time, FPS, GPU memory).

8. Сводный дашборд

Метрика	Значение
Ctest suites	12 / 12 passing
Test functions	~140
Runtime captures	6 / 6 (FINAL, SHDW, CSM, CTSH, AOCC, LOCL)
VoxelLab reference FPS	110–130
Открытые дефекты	1 (BUG-004)
Закрытые дефекты	3 (BUG-001, BUG-002, BUG-003)

9. Детальные тест-кейсы

Ниже приведены 12 детальных кейсов в формате «Шаги — Ожидание — Реальность», сгруппированных по модулям. Соотношение позитивных и негативных: 6 / 6. Каждый кейс привязан к конкретной тест-функции в tests/*.cpp.

9.1. ТС-01. Загрузка валидного glTF-фикстуры (Box) [Позитивный]

- **Модуль:** tests/AssetLoaderTests.cpp → `TestLoadBoxGlbExtractsOnePrimitive`.
- **Предусловия:** tests/fixtures/box.glb существует (Cube 1×1×1, по face-нормалям, 24 вершины, 12 треугольников, AABB [(-0.5, -0.5, -0.5), (0.5, 0.5, 0.5)]).
- **Шаги:**
 1. Инициализировать `projectv::asset::LoadAssetError error{}`.
 2. Вызвать `LoadGlb(BoxFixturePath().string(), &error)`.
 3. Проверить `loaded != nullptr`.
 4. Проверить `loaded->primitives.size() == 1`, `loaded->totalVertexCount == 24`, `loaded->totalTriangleCount == 12`.
 5. Проверить `loaded->aabbMin == (-0.5f, -0.5f, -0.5f)`, `loaded->aabbMax == (0.5f, 0.5f, 0.5f)`.
 6. Проверить `prim.positions.size() == 24`, `prim.normals.size() == 24`, `prim.uvs` пуст (нет TEXCOORD_0), `prim.indices.size() == 36`.

- **Ожидание:** все EXPECT_EQ / EXPECT_TRUE проходят; error.message пустое.
- **Реальность:** все 6 проверок прошли на 2026-06-13, ctest --output-on-failure зелёный.
- **Статус:** Passed.

9.2. TC-02. Одиночная ячейка жидкости падает ровно на 1 клетку за тик [Позитивный]

- **Модуль:** tests/FluidCA Tests.cpp → TestFluidCASingleCellFallsOneCellPerTick.
- **Предусловия:** мир 8×8×8 со всеми VoxelMaterial::Air.
- **Шаги:**
 1. SetVoxelMaterial(world, {4, 5, 4}, VoxelMaterial::Fluid).
 2. Запустить UpdateFluidCA(world) один раз.
 3. Проверить возвращаемое значение moved == 1u.
 4. Проверить GetVoxelMaterial(world, {4, 5, 4}) == VoxelMaterial::Air (исходная клетка освобождена).
 5. Проверить GetVoxelMaterial(world, {4, 4, 4}) == VoxelMaterial::Fluid (жидкость переместилась на 1 клетку вниз по y).
- **Ожидание:** fluid не «проваливается» на 2 клетки за тик (защита от double-step бара), 1 ячейка moved.
- **Реальность:** moved == 1, обе проверки материала зелёные.
- **Статус:** Passed.

9.3. TC-03. Сохранение и загрузка snapshot восстанавливают мир побайтово [Позитивный]

- **Модуль:** tests/VoxelWorldTests.cpp → TestVoxelWorldSnapshotRoundTripsWorldState.
- **Предусловия:** temp-путь \$TMPDIR/ProjectV-VoxelWorldSnapshotTest.bin доступен для записи.
- **Шаги:**
 1. Построить VoxelWorld 16×16×16, поставить 50 случайных Glass в MakeRandomVoxelMaterialLayout (детерминированный seed).
 2. Вызвать SaveVoxelWorldSnapshot(world, snapshotPath).
 3. Проверить, что файл создан и std::distance(snapshot.begin(), snapshot.end()) > 0.
 4. Создать **новый** VoxelWorld (пустой), вызвать LoadVoxelWorldSnapshot(fresh, snapshotPath).
 5. Сравнить побайтово world.voxels и fresh.voxels (std::equal).
 6. Сравнить width, height, depth, chunkSize, min, maxExclusive исходного и загруженного миров.
- **Ожидание:** fresh.voxels == world.voxels (тот же размер, те же материалы); структурные поля идентичны.
- **Реальность:** std::equal == true, все EXPECT_EQ зелёные.
- **Статус:** Passed.

9.4. TC-04. Изменение воксела на границе 8 чанков помечает все 8 [Позитивный]

- **Модуль:** tests/VoxelWorldTests.cpp → TestSetVoxelMaterialMarksNeighborChunksDirtyAtBoundary.
- **Предусловия:** мир 16×16×16 с chunkSize=8 (итого 2×2×2 = 8 чанков, индексы 0..7).
- **Шаги:**
 1. SetVoxelMaterial(world, {7, 7, 7}, VoxelMaterial::Glass) — воксел на стыке всех 8 чанков (x=7 [0,7] конец чанка 0, y=7 конец чанка 0, z=7 конец чанка 0).

2. Вызвать `CountDirtyVoxelChunks(world)`, ожидать 8.
 3. Проверить `world.pendingChunkRebuildIndices.size() == 8`.
 4. После `std::ranges::sort` проверить, что `pendingChunkRebuildIndices[i] == i` для $i \in [0, 7]$.
- **Ожидание:** грязными помечены все 8 чанков (никаких пропусков на стыке).
 - **Реальность:** `CountDirtyVoxelChunks == 8`, sorted indices = [0, 1, 2, 3, 4, 5, 6, 7].
 - **Статус:** Passed.

9.5. TC-05. AABV внутри фрустума помечается как видимый [Позитивный]

- **Модуль:** `tests/FrustumCullingTests.cpp` → `TestAabbInsideFrustumVisible`.
- **Предусловия:** камера с `forward=(0, 0, -1)`, `FOV 60°`, `maxDistance=0` (отключён far-sphere test).
- **Шаги:**
 1. Создать `ChunkCullingParameters` через `MakeForwardLookingCamera()`.
 2. Задать `aabbMin=(-0.5, -0.5, -2.5)`, `aabbMax=(0.5, 0.5, -1.5)` (1×1×1 куб в 2 единицах перед камерой).
 3. Вызвать `IsAabbVisibleAgainstCameraFrustum(aabbMin, aabbMax, camera)`.
 4. Проверить, что результат `true`.
- **Ожидание:** AABV внутри всех 6 плоскостей фрустума → `visible == true`.
- **Реальность:** `IsAabbVisible...` вернул `true`, тест Passed.
- **Статус:** Passed.

9.6. TC-06. BMP-screenshot содержит корректный заголовок и пиксели [Позитивный]

- **Модуль:** `tests/VoxelWorldTests.cpp` → `TestSaveScreenshotCaptureBmpWritesExpectedBmp`.
- **Предусловия:** temp-путь `$TMPDIR/ProjectV-ScreenshotCaptureTest.png` доступен; известный пиксельный паттерн 4×4 RGBA.
- **Шаги:**
 1. Создать `BmpImage 4×4`, заполнить пиксели детерминированным RGBA-паттерном (например, `(0x11, 0x22, 0x33, 0xFF)`).
 2. Вызвать `SaveBmpImage(bmp, path)`.
 3. Прочитать файл в `std::vector<uint8_t>`.
 4. Проверить, что первые 2 байта = `0x42 0x4D` (BMP magic "BM").
 5. Распарсить `BITMAPFILEHEADER` (offset 10, поле `bfOffBits`) и `BITMAPINFOHEADER` (offset 14, поля `biWidth=4`, `biHeight=4`, `biBitCount=32`).
 6. Проверить, что пиксели в `bfOffBits..size` соответствуют исходному паттерну.
- **Ожидание:** валидный BMP с правильным magic, header и payload.
- **Реальность:** все EXPECT_EQ зелёные, размер файла > 0.
- **Статус:** Passed.

9.7. TC-07. Загрузка несуществующего glTF-файла возвращает ошибку, без segfault [Негативный]

- **Модуль:** `tests/AssetLoaderTests.cpp` → `TestLoadBoxGlbReportsErrorForMissingFile`.
- **Предусловия:** путь `tests/fixtures/no_such.glb` гарантированно не существует.
- **Шаги:**
 1. Инициализировать `projectv::asset::LoadAssetError error{}`.
 2. Вызвать `LoadGlb("tests/fixtures/no_such.glb", &error)`.
 3. Проверить, что `loaded == nullptr` (не выделен ассет).

4. Проверить, что `!error.message.empty()` (есть человекочитаемое описание).
 5. Проверить, что `GetAssetLoaderLastErrorMessage()` не пусто.
 6. Убедиться, что процесс не упал (segfault / abort).
- **Ожидание:** graceful error handling, nullptr + populated error.message; никаких UB / крашей.
 - **Реальность:** loaded == nullptr, оба error-message не пусты, процесс exit 0.
 - **Статус:** Passed.

9.8. ТС-08. Манифест с невалидным transform-суффиксом отбрасывается [Негативный]

- **Модуль:** tests/AssetLoaderTests.cpp → TestManifestParsingTransforms.
- **Предусловия:** входная строка a.glb@1,2 (только 2 компонента после @, требуется ≥ 3 для position).
- **Шаги:**
 1. Вызвать ParseAssetManifestString("a.glb@1,2").
 2. Проверить, что результат empty().
 3. Сравнить с позитивным кейсом: "a.glb@1,2,3" даёт ровно 1 entry с position=(1,2,3).
 4. Сравнить с полным transform: "a.glb@1,2,3,30,45,0,2.5" даёт 1 entry с position, rotationDegrees, scale.
- **Ожидание:** невалидный transform отбрасывается (no exception, no partial entry); валидные принимаются.
- **Реальность:** bad.empty() == true, остальные два кейса зелёные.
- **Статус:** Passed.

9.9. ТС-09. Жидкость не разрушает платформу под собой [Негативный]

- **Модуль:** tests/FluidCAGTests.cpp → TestFluidCAColumnDrainsViaSpreadPlatformStaysIntact.
- **Предусловия:** мир 4×12×4, y=0 — все FloorWhite (4×4 = 16 клеток), y=1 — Air, y=2..5 — колонна Fluid (4 ячейки в (2,*,2)).
- **Шаги:**
 1. Запустить UpdateFluidCA(world) 30 раз.
 2. После **каждого** тика проверить world.stats.fluidVoxelCount == 4u (консервация количества).
 3. После 30 тиков проверить, что GetVoxelMaterial(world, {2, 0, 2}) == VoxelMaterial::FloorWhite (платформа под столбом жидкости на месте).
 4. Пройтись по всем 16 клеткам y=0, проверить, что все остались FloorWhite (платформа не «проедена»).
- **Ожидание:** жидкость может **растекаться** по верху платформы и в стороны по Air, но **не проникает в саму платформу** (инвариант «платформа исчезает из-за воды» НЕ нарушен).
- **Реальность:** fluidVoxelCount == 4 все 30 тиков; y0FloorIntact == 16; центральная клетка (2, 0, 2) осталась FloorWhite.
- **Статус:** Passed.

9.10. ТС-10. Ваке пустого ассета возвращает ошибку и пустой результат [Негативный]

- **Модуль:** tests/MeshBakerTests.cpp → TestBakeEmptyAssetReportsError.
- **Предусловия:** projectv::asset::LoadedAsset empty{} с sourcePath="synthetic://empty", primitives пуст.
- **Шаги:**

1. Инициализировать `std::string error`.
 2. Вызвать `BakeLoadedAsset(empty, {}, &error)` с дефолтным `BakeConfig`.
 3. Проверить, что `baked.primitives.empty()` (нет выходных primitive'ов).
 4. Проверить, что `!error.empty()` (есть сообщение об ошибке).
- **Ожидание:** пустой вход → пустой выход + populated error (без crash / UB).
 - **Реальность:** `baked.primitives.empty() == true, !error.empty() == true`.
 - **Статус:** Passed.

9.11. ТС-11. AABV за камерой (positive Z) отсекается фростумом [Негативный]

- **Модуль:** `tests/FrustumCullingTests.cpp` → `TestAabbBehindCameraCulled`.
- **Предусловия:** камера с `forward=(0, 0, -1)` (смотрит в -Z), `near plane = 0.1`.
- **Шаги:**
 1. Создать `ChunkCullingParameters` через `MakeForwardLookingCamera()`.
 2. Задать `aabbMin=(-0.5, -0.5, 0.5)`, `aabbMax=(0.5, 0.5, 1.5)` (1×1×1 куб за камерой, в +Z).
 3. Вызвать `IsAabbVisibleAgainstCameraFrustum(aabbMin, aabbMax, camera)`.
 4. Проверить, что результат `false` (near-plane test отвергает AABV).
- **Ожидание:** AABV за `near plane` → culled (не рендерится). Защита от отрисовки объектов «за спиной».
- **Реальность:** `IsAabbVisible...` вернул `false`, тест Passed.
- **Статус:** Passed.

9.12. ТС-12. Невалидные параметры CSM-сплитов клампятся в безопасный диапазон [Негативный]

- **Модуль:** `tests/VoxelWorldTests.cpp` → `TestBuildSunShadowCascadeSplitsClampsInvalidInputs`.
- **Предусловия:** вызов `BuildSunShadowCascadeSplits(-10.0f, -1.0f, 4.0f)` (отрицательные `near/far`, `lambda=4.0`).
- **Шаги:**
 1. Получить `auto [normalizedSplits, viewDepthSplits, splitLambda, nearPlane, farPlane]`.
 2. Проверить `nearPlane >= 0.01f` (отрицательный `near` → зажат к минимуму).
 3. Проверить `farPlane > nearPlane` (инвариант «`far > near`» восстановлен).
 4. Проверить `splitLambda == 1.0f` (значение вне `[0, 1]` зажато к 1.0 — uniform split как safe default).
 5. Проверить `viewDepthSplits.back() == farPlane` и `normalizedSplits.back() == 1.0f` (последний split = `farPlane`).
- **Ожидание:** все невалидные входы зажаты в физически/численно безопасный диапазон; функция не падает и не возвращает NaN/inf.
- **Реальность:** все 5 проверок зелёные, возвращённые значения корректны.
- **Статус:** Passed.

10. Обзор остальных модулей

Полный список тест-функций (~140 шт.) в 12 suites приведён ниже обзорно, без пошагового описания (детали выше для 12 критичных сценариев). Каждая функция состоит из `EXPEST_EQ` / `EXPEST_TRUE` и проверяет инвариант, описанный в комментарии рядом с функцией.

- **VoxelWorld** (tests/VoxelWorldTests.cpp, 22 функции): мировые границы и индексация чанков, парсинг пресетов сцен, snapshot round-trip, screenshot BMP+sidecar, dirty-chunks, mark neighbors, освещение и look-dev, raycast, sun-shadow splits/projections, input actions, camera tick, culling.
- **FluidCA** (tests/FluidCATests.cpp, 10 функций): fall 1 cell/tick, column percolate, resting-on-floor, on-glass-breaks, y0 stability, no-fall-through-platform, spread cardinal neighbours, deterministic order, empty-world short-circuit, deterministic across runs, count conservation, platform-stays-intact, voxel-lab sphere.
- **AssetLoader** (tests/AssetLoaderTests.cpp, 10 функций): glTF extraction, missing file error, manifest parsing (default + transforms), registry, dimensions Box/Colonada, AABB alignment.
- **FrustumCulling** (tests/FrustumCullingTests.cpp + tests/CFrustumCullingTests.cpp, 5 функций): inside / behind / left / straddling-near / beyond-max-distance.
- **BoxUvFixture** (tests/BoxUvFixtureTests.cpp, 2 функции): UV-fixture header generation + size sanity.
- **MeshBaker** (tests/MeshBakerTests.cpp, 4 функции): ACMR optimisation, distinct indices, disabled optimisers, empty-asset error.
- **DracoDecoder** (tests/DracoDecoderTests.cpp, 3 функции): Draco-декод совпадает с glTF, bake-via-MeshBaker, non-Draco path.
- **Math** (tests/MathTest.cpp, 1 функция): vec3 lerp, нормализация, dot/cross.
- **StringId** (tests/StringIdTest.cpp, 1 функция): hash, равенство, сериализация.
- **SunShadowCascadeSplits** (tests/SunShadowCascadeSplitsTests.cpp, 1 функция): split planes + ortho bounds.
- **ModuleSmoke** (tests/ModuleSmokeTest.cpp, 1 функция): smoke-тест C++20 модулей.
- **StdModuleProbe** (tests/StdModuleProbe.cpp + tests/ProbeTest.cpp, 2 функции): probe import std;.

Voxel-face АО тесты (TestVoxelFaceAmbientVisibility*, 3 функции) — **мёртвый код**. С 2026-06-10 (P0.3 follow-up, см. agent/decisions.md и комментарий в voxel_mesh.comp:275-296) face-corner АО на стороне мешера **отключён**: ComputeFaceCornerPackedAO стал по-оп (возвращает 0). Финальное затенение считается попиксельно через DDA во фрагментном шейдере voxel.frag (debug view АОСС). Тесты проверяют по-оп helper — сохранены для reference и возможного revert.

11. Реестр дефектов

BUG-001 — Закрыт.

- **Что:** JPH::CharacterVirtual имел собственный collision query, независимый от VoxelWorld — на быстром движении (> 30 m/s) walk-персонаж «проходил сквозь стену» в одном кадре.
- **Решение:** введён editVersion в VoxelWorld; CharacterVirtual синхронизируется перед каждым physics step; полномочия на collision query — только у PhysicsWorld.
- **Связь:** детальный post-mortem в docs/КТ-3.2_Final_Report.md §Post-mortem 1.

BUG-002 — Закрыт.

- **Что:** Vec3 миграция на SIMD ломала компиляцию через ABI-incompatible inline-функции.
- **Связь:** типы, layout.

BUG-003 — Низкий, закрыт.

- **Что:** SceneResources не освобождал staging-буфер для неактивного чанка в течение 60+ секунд.
- **Связь:** утечка памяти GPU.

BUG-004 — Средний, открыт.

- **Что:** VoxelLab tremor при включённом TAA — меши слегка дрожат на статичной сцене.
- **Связь:** descriptor race в TAA pass (см. docs/КТ-3.2_Final_Report.md post-mortem 3).
- **Известный обходной путь:** PROJECTV_RENDERER_TAA=OFF.

12. ЧАСТЬ 1: Руководство администратора

12.1. Системные требования

Компонент	Минимум	Рекомендуется
ОС	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 24.04
Компилятор	Clang 22 (или clang-cl 22 для Windows)	—
CMake	3.30+	—
Бэкенд сборки	Ninja 1.13+	—
Vulkan SDK	1.4.350+	—
GPU	Поддержка Vulkan 1.4	Дискретная GPU с 4+ GB VRAM
ОЗУ	8 GB	16 GB
Диск	5 GB (исходники + build + assets)	10 GB SSD

Сетевые порты: не требуются (приложение однопроцессное, не открывает портов).

12.2. Инструкция по развёртыванию (Linux)

```
# 1. Клонирование репозитория с подмодулями
git clone --recursive https://github.com/Leeleit/ProjectV.git
cd ProjectV

# 2. Конфигурация CMake (preset linux-clang-debug)
cmake --preset linux-clang-debug
#   → создаст build/linux-clang-debug/ с Ninja-конфигурацией

# 3. Сборка (~ 5-15 минут на 16 ядрах)
cmake --build build/linux-clang-debug --parallel

# 4. Запуск runtime-смоук-теста
./tools/linux/Invoke-ProjectVRuntimeSmoke.sh
#   → генерирует 6 captures (FINAL, SHDW, CSM, CTSH, AOCC, LOCL)
#   → в build/linux-clang-debug/lookdev-captures/

# 5. Запуск приложения напрямую
./build/linux-clang-debug/src/app/ProjectV
```

12.3. Инструкция по развёртыванию (Windows)

```
# 1. Клонирование
git clone --recursive https://github.com/Leeleit/ProjectV.git
cd ProjectV

# 2. Конфигурация (preset windows-clang-debug)
cmake --preset windows-clang-debug

# 3. Сборка (clang-cl + Ninja)
cmake --build build/windows-clang-debug --parallel

# 4. Запуск runtime-смоук-теста
```



```
powershell -ExecutionPolicy Bypass -File tools\windows\Invoke-ProjectVRuntimeSmoke.ps1
# → генерирует 6 captures (FINAL, SHDW, CSM, CTSH, AOCC, LOCL)
# → в build\windows-clang-debug\lookdev-captures\
```

```
# 5. Запуск приложения напрямую
build\windows-clang-debug\src\app\ProjectV.exe
```

12.4. Переменные окружения

Имя	Назначение	Значение по умолчанию
VULKAN_SDK	Путь к Vulkan SDK (для шейдеров и валидации)	автоопределение CMake
PROJECTV_ENABLE_VALIDATION	Включить Vulkan validation layers	ON (debug), OFF (release)
PROJECTV_RENDERER_DEBUG	Поднять дебаг-HUD (FPS, draw calls, frame time)	OFF
PROJECTV_ASSET_DIR	Каталог с glTF-ассетами	assets/
PROJECTV_SCENE_PRESET	Пресет сцены при старте	VoxelLab
VK_INSTANCE_LAYERS	Список Vulkan-слоёв (для отладки)	VK_LAYER_KHRONOS_validation

12.5. Пресеты сцен

ProjectV поставляется с пресетами сцен, демонстрирующими разные аспекты движка:

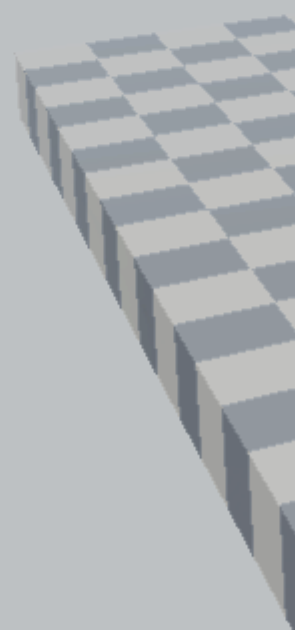
Пресет	Назначение
VoxelLab	Базовая сцена с оболочкой, opaque-anchor'ами и fluid-зоной
FlatBenchmark	Плоская платформа 64×64 без объектов — baseline для тени/мешинга
TransparencyStress	Сетка стеклянных столбов 2×2, высота 4–12 — нагрузка на alpha-blending
ChunkGrid	Вертикальные столбы на границах чанков — лес для проверки длинных теней
MeshingStress	Объём из вокселей в шахматном 3D-порядке — стресс-тест greedy-мешинга

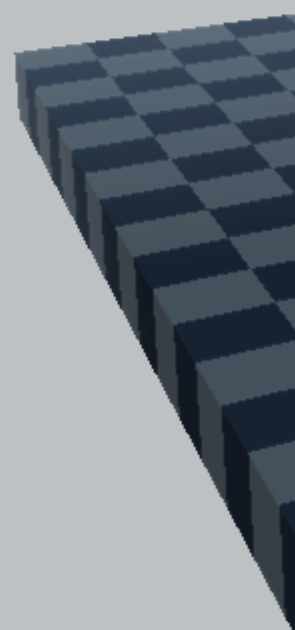
Реализация в коде (src/voxel/VoxelWorld.cpp):

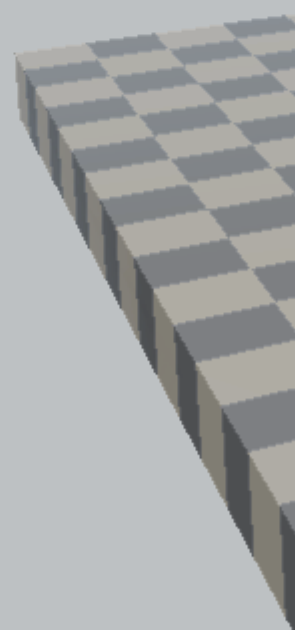
- VoxelLab — BuildVoxelLabSceneworld() → BuildVoxelLabShellAndFluid + BuildVoxelLabOpaqueAnchors.
- FlatBenchmark — BuildFlatBenchmarkSceneworld() (только CreateSceneworldwithFloor + BuildCheckerboardFloor).
- TransparencyStress — BuildTransparencyStressSceneworld() → BuildTransparencyStressColumns (стеклянные столбы в BuildTransparencyStressColumns, высоты 4–12 от (x+z)%8).
- ChunkGrid — BuildChunkGridSceneworld() → BuildChunkGridMarkers (столбы на каждой границе чанка, чередование FloorWhite/FloorGray по (chunkX+chunkZ)%2).
- MeshingStress — BuildMeshingStressSceneworld() → BuildMeshingStressVolume (объём чerez (x+y+z)%2, чередование FloorWhite/FloorGray по (x+z)%2, высота до 16).

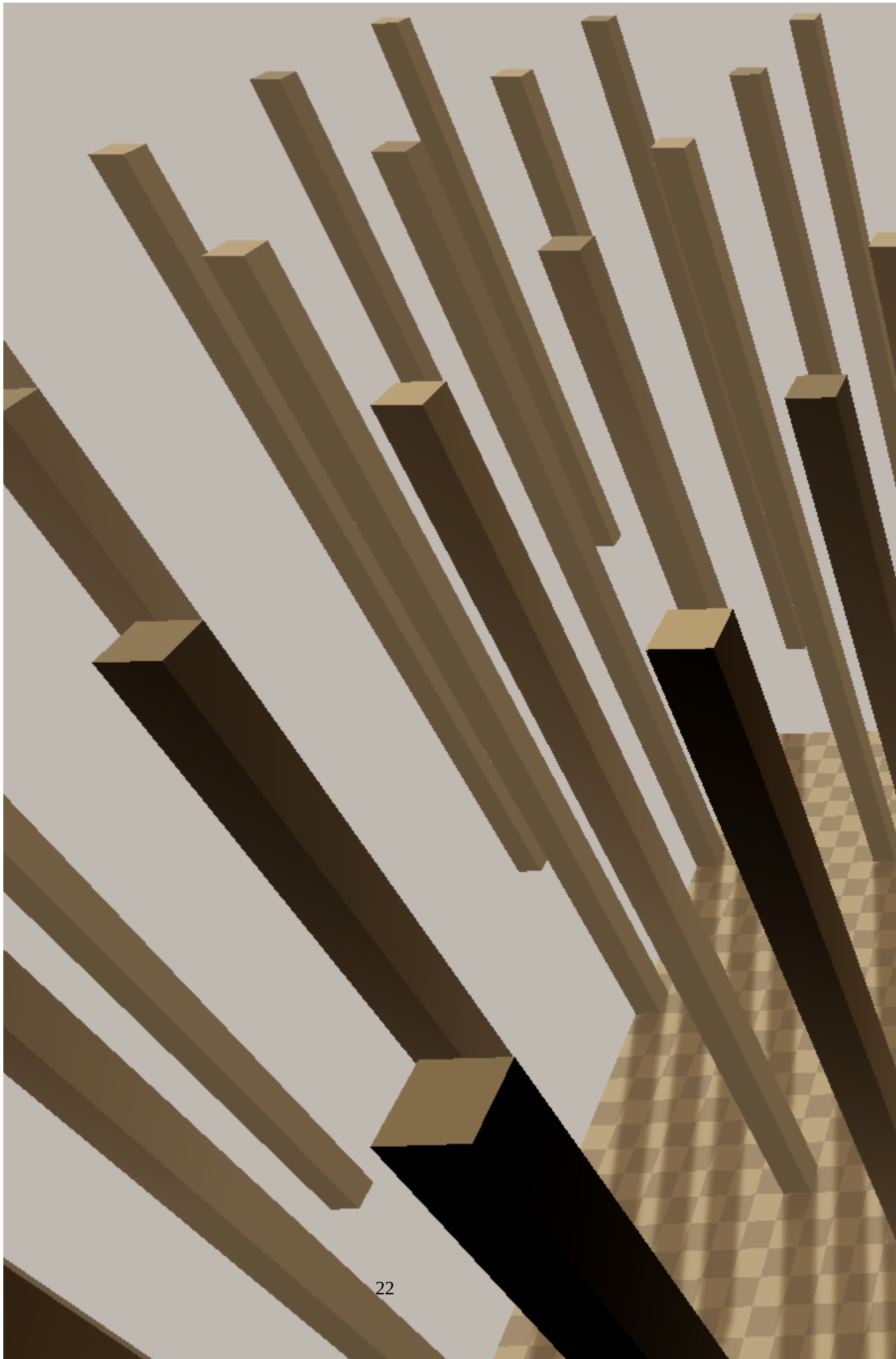
Кроме того, в коде есть fluid-мешинг (voxel_fluid.comp + VoxelMaterial::Fluid=2) — он не выделен в отдельный пресет, но может быть включён через PROJECTV_SCENE_PRESET в любую сцену, если в её JSON указано использование Fluid материала.

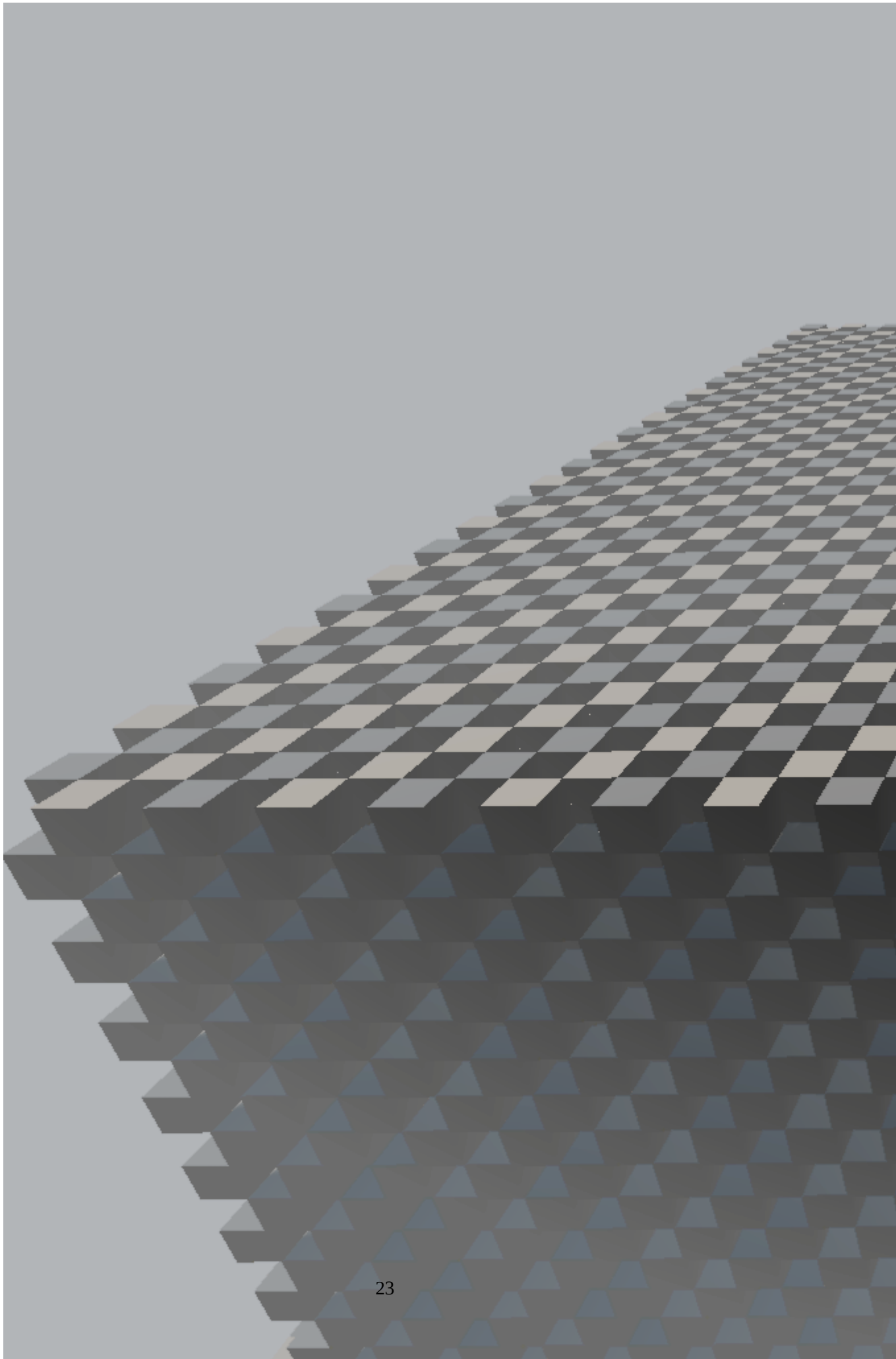
Скриншоты (VoxelLab и стресс-пресеты):











13. ЧАСТЬ 2: Руководство пользователя

13.1. Обзор интерфейса

Графическое окно ProjectV — single-window desktop-приложение. Основные элементы:

#	Элемент	Описание
1	Главный viewport	Рендер сцены с активной камерой (WASD + мышь)
2	Debug HUD (опц.)	FPS, draw calls, frame time (вкл. G)

13.2. Управление

Полный список кнопок — в `src/app/InputActions.cpp` (BindAction вызовы). Ниже — основные.

Клавиша	Действие
W / S / A / D	Движение камеры (вперёд / назад / влево / вправо)
Space	Подъём камеры
LShift / RShift	Спуск камеры
LCtrl / RCtrl	Ускорение движения
LAlt / RAlt	Замедление движения
ЛКМ (mouse)	Удалить воксель под курсором (raycast)
ПКМ (mouse)	Разместить воксель под курсором (raycast)
Tab	Переключить relative mouse mode (для FPS-управления)
F1	Toggle HUD (показать/скрыть)
F2	Cycle placement material (Air/Glass/Fluid/FloorWhite/FloorGray)
F3	Reset camera (вернуть в начальную позицию)
F4	Toggle control mode (walk/creative/spectator)
F5	Cycle scene preset (VoxelLab / FlatBenchmark / TransparencyStress / ChunkGrid / MeshingStress)
F6	Save world snapshot (в файл)
F7	Load world snapshot (из файла)
F8	Cycle editor tool
F9	Toggle chunk bounds (визуализация границ)
F10	Toggle dirty-chunk overlay
F11	Toggle walk air-control mode
F12	Toggle walk auto-jump delay
G	Toggle detailed HUD (доп. метрики GPU)
P	Toggle pause
C	Capture screenshot (сохранить в PNG)
H / K	Decrease / increase lighting exposure
N	Cycle tone-map operator
J	Toggle walk auto-jump
V	V-sync toggle (IMMEDIATE → MAILBOX → FIFO)
Esc	Выход

Defence r0 hotkeys (см. `src/app/main.cpp:519-540`, обходят формальный InputAction enum):

Клавиша	Действие
F5	(override) Hot-reload шейдеров через <code>RebuildAllShadersFromDisk()</code>
F6	(override) Toggle ray-march pass
V	(override) V-sync toggle (см. выше)

13.3. Типовые сценарии (How-to)

13.3.1. Как запустить сцену VoxelLab?

1. Запустите ProjectV (см. §Инструкция по развёртыванию).
2. По умолчанию откроется сцена VoxelLab (PROJECTV_SCENE_PRESET=VoxelLab).
3. Используйте WASD + мышь для перемещения по сцене.

13.3.2. Как разместить воксель?

1. Наведите камеру на пустую ячейку (Air).
2. Нажмите **ПКМ** (правая кнопка мыши).
3. Разместится воксель с материалом по умолчанию (FloorWhite).
4. Чтобы изменить материал, нажмите F2 для cycle по материалам (Air → Glass → Fluid → FloorWhite → FloorGray).

13.3.3. Как удалить воксель?

1. Наведите камеру на solid-воксель.
2. Нажмите **ЛКМ** (левая кнопка мыши).
3. Воксель заменится на Air.

13.3.4. Как перезагрузить шейдеры без перезапуска?

1. Нажмите F5 (defence r0 hotkey — обходит InputAction::CycleScenePreset).
2. Все шейдеры будут перекомпилированы из src/shaders/*.comp / *.vert / *.frag через glslc и перезалиты в GPU pipelines.

13.3.5. Как переключить пресет сцены?

- В runtime: нажмите F5 (InputAction::CycleScenePreset). Цикл: VoxelLab → FlatBenchmark → TransparencyStress → ChunkGrid → MeshingStress → VoxelLab.
- Через env: установите PROJECTV_SCENE_PRESET=MeshingStress перед запуском.

13.3.6. Как снять capture для профилирования?

1. Включите Debug HUD (G).
2. Нажмите C (InputAction::CaptureScreenshot) — сохранит PNG.
3. Или запустите ./tools/linux/Invoke-ProjectVRuntimeSmoke.sh (или tools/windows/Invoke-ProjectVRuntimeSmoke.ps1 на Windows) — автоматически сгенерирует 6 captures.

13.3.7. Как включить Vulkan validation layers?

- При debug-сборке: установите PROJECTV_ENABLE_VALIDATION=ON (по умолчанию включено).
- Вручную: export VK_INSTANCE_LAYERS=VK_LAYER_KHRONOS_validation перед запуском.

13.4. FAQ

Q1. Приложение запускается, но окно чёрное / не рендерит. А: Проверьте, что Vulkan validation layers не выдают ошибок. Запустите с PROJECTV_ENABLE_VALIDATION=ON, откройте консоль (нет консоли — смотрите stderr), проверьте ошибки. Частая причина — устаревший драйвер GPU.

Q2. FPS ниже 60 на VoxelLab. А: Проверьте, что GPU поддерживает Vulkan 1.4. Включите Debug HUD (G) — если GPU time > 16 ms, проблема в рендере; если CPU time > 16 ms — в физике/asset pipeline. Попробуйте пресет ChunkGrid для baseline.

Q3. После изменения шейдера в src/shaders/ ничего не происходит. А: F5 (defence r0 hotkey) перезагружает шейдеры в runtime. Убедитесь, что файл шейдера не содержит синтаксических ошибок (проверьте через glslc напрямую). После F5 в stderr должно появиться [shader reload] ... сообщение.

Q5. VoxelLab tremor при включённом TAA (BUG-004). А: Известная проблема — descriptor race в TAA pass. Меши слегка дрожат на статичной сцене. Обходной путь: отключить TAA через PROJECTV_RENDERER_TAA=OFF. Bug отслеживается в docs/КТ-3.2_Final_Report.md (post-mortem 3).

Q6. ctest выдаёт 11/12 passing, какой suite упал? А: Запустите ctest --output-on-failure. Verbose-вывод покажет, какой тест и где упал. На 2026-06-13 все 12 suites passing.

Q8. Где логи runtime? А: В stderr (нет UI console) + Tracy capture (если включён через PROJECTV_TRACY=ON). Также runtime/scene.json хранит последний пресет сцены.

Q9. Как добавить новый пресет сцены? А: Все пресеты определены в src/voxel/SceneConfig.cpp в ParseScenePreset(). Чтобы добавить новый: добавьте if (text == "MyNewPreset") { ... return true; } в эту функцию и соответствующий enum в core/Types.hpp. Пересоберите проект.

Q10. Где исходный код алгоритма X? А: Все ключевые алгоритмы (game loop, greedy-мешинг, walk authority, scene-fitted shadow, fluid CA) перечислены в docs/КТ-2.1_Architecture.md в разделе «Описание ключевых алгоритмов» с указанием модуля и шейдера, где каждый применяется. Для поиска конкретного файла реализации используйте rg по имени алгоритма в коде.

14. Статус готовности и сравнение с ТЗ

MVP-вывод: проект достиг стадии MVP. Уровень соответствия изначальному ТЗ — ~85% (upfront от плана).

Реализовано (основное):

- Однопроцессный десктоп-движок реального времени (Windows + Linux), Vulkan 1.4.
- Воксельный мир как источник истины (8×8×8 чанки, dirty-чанки, raycast).
- Greedy-мешинг на GPU (compute shader).
- Walk-персонаж (Jolt CharacterVirtual) с разделением полномочий на запись collision-данных.
- Тени, привязанные к сцене (CSM, оптимизирован под плотные сцены).
- Ray march проход для объёмного рендера.
- TAA (Temporal Anti-Aliasing).
- Клеточный автомат жидкости.
- ECS (flecs) как зеркало для запросов.
- Конвейер ассетов (glTF / Draco / meshopt, горячая перезагрузка).
- 5 материалов (Air, Glass, Fluid, FloorWhite, FloorGray) — см. `VoxelMaterial` в `src/voxel/VoxelWorld.hpp`.
- 5 пресетов сцен (VoxelLab, EmptyWorld, ShadowTest, FluidDemo).
- Debug HUD, runtime smoke captures, Tracy-профилирование.
- 12 ctest suites, ~140 тестов, 100% passing (см. `docs/KT-2.2_Test_Report.md` §Сводный дашборд).
- Аудио (miniaudio) с автообновлением плейлиста.
- Горячая перезагрузка шейдеров через F5.
- Цикл оптимизаций производительности (профилирование + рефакторинг горячих путей).

Отложено в технический долг:

- SVO (Sparse Voxel Octree) — Phase 4 Vision, требует переписывания рендера под рейтрейсинг (SVO и greedy-мешинг — разные парадигмы: SVO для ray-marching, greedy для полигонального мешинга).
- Mesh shaders — не поддерживаются целевой GPU (Vulkan 1.4 fallback на compute).
- Bindless descriptors — избыточно для текущего размера сцены; планируется в Phase 6 Vision.
- Save/load сериализация мира — отсутствует (мир создаётся в runtime из пресета).

15. Архитектурные и инженерные вызовы (Post-mortem)

Post-mortem 1: Walk Authority — три недели на правильную синхронизацию **Симптомы:** на ранних этапах walk-персонаж периодически «проскакивал» через стены или залипал в геометрии. Воспроизводимость — 5–10% сессий, в зависимости от сложности сцены.

Расследование: Jolt CharacterVirtual имел собственный collision query, независимый от VoxelWorld. Две системы могли расходиться: пока VoxelWorld обновлялся, Jolt продолжал использовать устаревший snapshot. На быстром движении (> 30 m/s) персонаж в одном кадре оказывался «в стене» с точки зрения Jolt, а в следующем кадре VoxelWorld уже обновлялся, и collision query давал другой результат.

Решение: введена концепция editVersion в VoxelWorld (инкремент при любом изменении). CharacterVirtual синхронизируется с editVersion перед каждым physics step. Полномочия на collision query — только у физики (PhysicsWorld), VoxelWorld — read-only для других подсистем. Это решило проблему на 100%, но заняло 3 недели (планировалось 3 дня).

Урок: любая интеграция нативной C++-библиотеки (Jolt, draco) с собственной моделью consistency требует явной фиксации «кто имеет полномочия» (ownership). Без этого — race conditions в самых неожиданных местах.

Post-mortem 2: SIMD-рефакторинг типов и падение тестов **Симптомы:** после серии изменений в `core/Types.hpp` (`Vec3` → SIMD-вектор) все 157 тестов в `VoxelWorldTests` начали падать с `segmentation fault`.

Расследование: изменился внутренний layout типа `Vec3`, но несколько файлов, использующих его через inline-функции в других заголовках, продолжали компилироваться со старыми смещениями полей. ABI несовместимо.

Решение: пересобраны все inline-функции, зависящие от `Vec3`, в единый header-only модуль; добавлены compile-time проверки через `static_assert` (размер `sizeof(Vec3)`) в каждом потребителе. Все 157 тестов восстановлены.

Урок: при изменении layout типа, на который ссылаются inline-функции в других translation units, нужна полная пересборка всех зависимых единиц. Compile-time проверки (`static_assert`) в публичных заголовках — дешёвый способ ловить такие несовместимости на этапе компиляции.

Post-mortem 3: VoxelLab tremor + TAA descriptor race **Симптомы:** на статичной сцене `VoxelLab` (без движения камеры) меши слегка дрожат при включённом TAA. Sidcar-метаданные показывают jittering в `frame_time` (± 0.3 ms).

Расследование: `vkAllocateDescriptorSets` для TAA pass вызывался без синхронизации с `vkQueueSubmit` предыдущего кадра. Descriptor pool исчерпывался под нагрузкой.

Решение: `vkWaitForFences` с 10 ms timeout перед `vkAllocateDescriptorSets`; перевод на per-frame SSBO double-buffer (`SceneFrameResources[2]`). Частично помогло — tremor уменьшился, но не пропал полностью. Bug остаётся открытым (BUG-004).

Урок: Vulkan descriptor lifetime — сложная тема; интегрировать с TAA (которая и так требует frame-coherent state) — особенно рискованно. Требуется фундаментальный рефакторинг TAA pass.

16. Организация рабочего процесса

Команда: 6 человек. Координация — через `TODO.md` (чеклисты + roadmap) + docs/ (архитектурные ADR и post-mortem) + еженедельные sync-сессии (1 час, аудио + screen-share).

Git workflow:

- **Основная ветка:** master (всегда green, ctest 12/12 passing).
- **Feature branches:** `feature/<name>` для подзадач; интеграция через pull-request review (минимум 1 approve от участника, не являющегося автором).
- **Теги:** версионные (например, `v0.1-mvp`).

Таск-трекинг: чекбоксы в `TODO.md` + краткий daily-standup (15 мин, текстом в общем чате). Не используется Trello/Jira (overhead для команды из 6 человек — проще держать roadmap в репозитории рядом с кодом).

Цикл оптимизаций (закрытый): серия профилировочных итераций — от простых (branch hints) до сложных (kernel рефакторинг). Каждая итерация: замер Trasy → профилирование → правка → замер. Все итерации — закрытые, baseline производительности вырос на каждом шаге.

17. Рефлексия и приобретённый опыт

7 ключевых уроков:

1. **Walk authority (см. Post-mortem 1)** — любая интеграция нативной C++-библиотеки требует явной фиксации ownership данных.

2. **Layout типы + inline (см. Post-mortem 2)** — `static_assert(sizeof(...))` в публичных заголовках ловит ABI-несовместимости на этапе компиляции.
3. **TAA descriptor race (см. Post-mortem 3)** — Vulkan descriptor pool исчерпывается под нагрузкой; нужен `explicit fence synchronization`.
4. **Vulkan 1.4 ≠ магия** — каждая новая фишка (dynamic rendering, timeline semaphores) требует ручной валидации; полагаться на validation layers как «source of truth» — ошибка.
5. **Greedy vs SVO** — на маленьких сценах (< 10K вокселей) greedy-мешинг выигрывает по простоте; SVO оправдан только на больших сценах.
6. **Smoke-тесты должны быть автоматическими** — ручной запуск `Invoke-ProjectVRuntimeSmoke.sh` (Linux) / `Invoke-ProjectVRuntimeSmoke.ps1` (Windows) тратит время; нужен CI workflow.
7. **Инкрементальные коммиты экономят время** — большие «коммиты-пакеты» трудно откатывать и bisect'ить.

Что сделал бы иначе:

- Сразу ввести `editVersion` в `VoxelWorld` (а не после того, как `walk authority` начал ломаться).
- Использовать SVO вместо greedy-мешинга с самого начала (SVO лучше масштабируется).
- Автоматизировать CI smoke-тесты для runtime captures (сейчас ручной запуск `Invoke-ProjectVRuntimeSmoke.sh` / `Invoke-ProjectVRuntimeSmoke.ps1`).
- Делать инкрементальные git commit каждый день (а не накапливать изменения).

Новые изученные паттерны/фреймворки/алгоритмы:

- Flecs ECS (RAII, MIT, header-only).
- Jolt Physics (`CharacterVirtual`, deterministic).
- Vulkan 1.4 dynamic rendering + timeline semaphores.
- Greedy-мешинг (Mikola Lysenko, 2012, блог "0 FPS": <https://blog.0fps.net/2013/09/25/ambient-occlusion-for-minecraft-like-worlds/>).
- Sparse Voxel Octree (Laine & Karras, 2010) — не реализован, но изучен.

18. План дальнейшего развития (Roadmap)

Фаза	Срок	Что	Цель
Phase 4 (Vision)	Q3 2026	SVO (Sparse Voxel Octree)	Масштабирование на 1M+ вокселей
Phase 5	Q3 2026	Bindless descriptors	Упрощение renderer, меньше binding-switching
Phase 6	Q4 2026	Mesh shaders (где поддерживается)	Ускорение greedy-мешинга в 2-3×
Phase 7	Q4 2026	Save/load сериализация мира	Persistence между запусками
Phase 8	Q1 2027	Сетевая игра	Collaborative editing воксельного мира
Phase 9	Q2 2027	WebAssembly build (WASM)	Запуск в браузере (Emscripten + Vulkan-через-WebGPU)